

Trabajo Práctico Especial



Proxy SOCKS5

Protocolos de Comunicación 2026

Grupo 2

Tiago Heras - 65627

Mateo Arias - 65613

Amador Vallejo Tapia - 65454

Manuel García Puente - 65505

Instituto Tecnológico de Buenos Aires

14 de julio de 2026

Índice

1. Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas	4
1.1. Arquitectura general del servidor proxy SOCKS5	4
1.2. Protocolo de monitoreo y configuración	4
1.2.1. Descripción general	4
1.2.2. Decisiones de diseño	4
1.2.3. Formato de los mensajes	5
1.2.4. Comandos soportados	5
1.2.5. Flujo de una sesión típica	6
1.3. Aplicación cliente de monitoreo	6
1.4. Métricas y registro de accesos	7
2. Problemas encontrados durante el diseño y la implementación	7
3. Limitaciones de la aplicación	7
3.1. Capacidad del selector y file descriptors	7
3.2. Backlog de <code>listen</code> vs. clientes conectados	8
3.3. Registro de accesos síncrono	8
3.4. Alcance del protocolo SOCKS	8
4. Posibles extensiones	8
5. Conclusiones	8
6. Ejemplos de prueba	9
6.1. Prueba básica del proxy SOCKS5	9
6.2. Prueba de monitoreo	9
6.3. Prueba de graceful shutdown	9
6.4. Pruebas de estrés	9
7. Guía de instalación	11
7.1. Requisitos	11
7.2. Compilación	11
8. Instrucciones para la configuración	11
9. Ejemplos de configuración y monitoreo	12
9.1. Escenario 1: monitoreo local	12
9.2. Escenario 2: auditoría de accesos	12
10. Documento de diseño del proyecto	12
10.1. Diagrama de módulos	12
10.2. Componentes propios	13
10.2.1. Máquina de estados SOCKS5 (<code>socks5nio.c</code>)	13
10.2.2. Parsers SOCKS5	13
10.2.3. Servidor de monitoreo (<code>mng_server.c</code>)	13
10.2.4. Cliente de monitoreo (<code>client.c</code>)	13

10.3. Flujo de una conexión SOCKS5	13
--	----

1 Descripción detallada de los protocolos diseñados y aplicaciones desarrolladas

1.1 Arquitectura general del servidor proxy SOCKS5

El servidor proxy SOCKS5 está implementado como una aplicación monohilo que utiliza multiplexación de entrada/salida no bloqueante mediante un selector de eventos. Todas las conexiones entrantes y salientes se manejan en un único hilo de ejecución, con la única excepción de la resolución de nombres de dominio (FQDN), que se delega a un hilo auxiliar mediante `getaddrinfo(3)`.

Cada conexión SOCKS5 se modela como una máquina de estados que transiciona a través de las siguientes fases:

```
HELLO_READ -> HELLO_WRITE -> [AUTH_READ -> AUTH_WRITE] -> REQUEST_READ
-> REQUEST_RESOLV -> REQUEST_CONNECTING -> REQUEST_WRITE -> COPY
-> DONE / ERROR
```

- **HELLO**: el cliente ofrece métodos de autenticación; el servidor elige `NO AUTH` si no hay usuarios registrados, o usuario/contraseña (RFC 1929) en caso contrario.
- **AUTH**: si corresponde, se validan las credenciales contra la tabla en memoria (hasta 10 usuarios, configurables por CLI o en caliente).
- **REQUEST**: solo se soporta `CONNECT`. Se aceptan destinos IPv4, IPv6 y FQDN. Si un FQDN resuelve a varias direcciones, se intentan en orden hasta lograr conectar.
- **COPY**: túnel bidireccional. Tras cada lectura exitosa se intenta un `write` inmediato al peer (`select` → `read` → `write`); si no entra todo, el resto queda en el buffer y se espera `OP_WRITE`.

1.2 Protocolo de monitoreo y configuración

1.2.1. Descripción general

El protocolo de monitoreo es un protocolo de texto sobre TCP, independiente de SOCKS5, expuesto en un socket pasivo propio (por defecto `127.0.0.1:8080`). Permite consultar métricas y modificar usuarios o el secreto de administración sin reiniciar el servidor.

1.2.2. Decisiones de diseño

- **Texto plano**: facilita depuración y una especificación estilo RFC sin serialización binaria.
- **Una línea por comando / una línea por respuesta**: el estado por conexión es mínimo y encaja en la misma STM del selector.
- **Autenticación por secreto compartido**: cada comando comienza con un token secreto configurado con `-A` (y cambiable en runtime con `SET_SECRET`). Sin el secreto correcto, la respuesta es `-ERR unauthorized`.

- **Terminadores:** los comandos aceptan LF o CRLF (el CR opcional se ignora). Las respuestas siempre se emiten con CRLF. Se eligió esta asimetría a propósito: ser permisivos al parsear (compatible con clientes que mandan solo \n) y estrictos al responder (línea bien formada para parsers que esperan CRLF).

1.2.3. Formato de los mensajes

```

1 comando      = secreto SP verbo *(SP argumento) EOL
2 secreto      = 1*(%x21-7E)
3 verbo        = 1*ALPHA
4 argumento    = 1*(%x21-7E)
5 EOL          = [CR] LF
6 CR           = %x0D
7 LF           = %x0A
8 SP           = %x20

```

Listing 1: Gramática ABNF de comandos

```

1 respuesta    = status [SP datos] CRLF
2 status       = "+OK" / "-ERR"
3 datos        = 1*(%x20-7E)
4 CRLF         = CR LF

```

Listing 2: Gramática ABNF de respuestas

1.2.4. Comandos soportados

1. GET_METRICS

- Sintaxis: <secreto>GET_METRICS\n
- Respuesta: +OK ACT:<n>HIST:<n>BYTES:<n>\r\n
- ACT: conexiones SOCKS5 activas (ya aceptadas y en curso).
- HIST: conexiones históricas desde el arranque.
- BYTES: bytes transferidos en el túnel (ambos sentidos).

2. ADD_USER

- Sintaxis: <secreto>ADD_USER <usuario><contraseña>\n
- Hay un espacio entre el usuario y la contraseña.
- Respuesta: +OK\r\n o -ERR <motivo>\r\n
- Máximo 10 usuarios; cada campo hasta 255 caracteres.

3. DEL_USER

- Sintaxis: <secreto>DEL_USER <usuario>\n
- Respuesta: +OK\r\n o -ERR user not found\r\n

4. SET_SECRET

- Sintaxis: <secreto>SET_SECRET <nuevo_secreto>\n

- Cambia el secreto de management en caliente.
- Respuesta: +OK\r\n

1.2.5. Flujo de una sesión típica

```

1 Cliente:                               Servidor:
2 |  CONECTAR (TCP)                       |
3 | ----->|
4 |  changeme GET_METRICS\n              |
5 | ----->|
6 |                                     | +OK ACT:3 HIST:150 BYTES
7 |                                     | :1048576\r\n
8 | <-----|
9 |  changeme ADD_USER alberto pass123\n |
10 | ----->|
11 |                                     | +OK\r\n
12 | <-----|
13 |  changeme DEL_USER alberto\n         |
14 | ----->|
15 |                                     | +OK\r\n
16 | <-----|
17 |  CERRAR CONEXIÓN                     |
18 | ----->|

```

Listing 3: Ejemplo de sesión de monitoreo

1.3 Aplicación cliente de monitoreo

El cliente es una CLI con I/O bloqueante que abstrae el protocolo: arma el comando con el secreto (-A), envía una línea, lee la respuesta hasta CRLF y presenta un mensaje legible. No vuelca el wire format a stdout.

```

1 ./bin/client [opciones] <comando>
2
3 Opciones:
4   -L <dirección>    IP del servidor de management (defecto:
5                       127.0.0.1)
6   -P <puerto>      Puerto de management (defecto: 8080)
7   -A <secreto>       Secreto de management (defecto: changeme)
8
9 Comandos:
10  get-metrics
11  add-user <usr> <pass>
12  del-user <usr>
13  set-secret <nuevo_secreto>

```

Listing 4: Uso del cliente de monitoreo

```

1 $ ./bin/client -A changeme get-metrics
2 conexiones activas: 0
3 conexiones historicas: 12

```

```
4 bytes transferidos: 409600
5
6 $ ./bin/client -A changeme add-user juan pass123
7 usuario agregado
```

Listing 5: Ejemplos del cliente

Ante `-ERR` o fallo de red, el proceso termina con código de salida distinto de 0.

1.4 Métricas y registro de accesos

Las métricas `ACT`, `HIST` y `BYTES` son volátiles (se pierden al reiniciar). Los accesos exitosos al origen se registran en `stdout` con formato:

```
[YYYY-MM-DD HH:MM:SS] User: <usr> | FQDN: <nombre|-> | IP Destino: <ip:port>
```

El administrador puede redirigir la salida del servidor a un archivo (`./bin/server ... >access.log`).

2 Problemas encontrados durante el diseño y la implementación

- **Resolución DNS no bloqueante:** `getaddrinfo(3)` bloquea. Se ejecuta en un hilo auxiliar y se notifica al selector con `selector_notify_block`. Hubo que cuidar el ciclo de vida del `struct socks5` (no liberarlo mientras el hilo DNS aún corre) y el acceso a `origin_resolution`.
- **Múltiples direcciones por FQDN:** si la primera IP falla, hay que recorrer el resto de `addrinfo` sin perder el estado de la STM ni dejar fds huérfanos.
- **Graceful shutdown:** ante `SIGTERM`/`SIGINT` se deja de aceptar conexiones nuevas y se espera a que las activas terminen. El desafío fue coordinar el socket pasivo (dejar de escucharlo) con el contador de conexiones vivas sin cortar túneles en `COPY`.
- **Latencia extra en el túnel:** el patrón inicial `select` $\rightarrow$ `read` $\rightarrow$ `select` $\rightarrow$ `write` introducía un round-trip de selector innecesario. Se corrigió con `write` optimista inmediato tras cada lectura exitosa.
- **Autenticación dinámica:** pasar de “sin usuarios = NO AUTH” a “hay usuarios = exigir RFC 1929” en caliente (vía `ADD_USER`) obliga a que la selección de método `HELLO` consulte el estado actual de la tabla, no una política fija de arranque.

3 Limitaciones de la aplicación

3.1 Capacidad del selector y file descriptors

El selector se crea con capacidad fija (1024 descriptores). Cada túnel SOCKS usa dos fds (cliente y origen). En pruebas de estrés, 500 conexiones concurrentes se establecen correctamente; por encima de ese orden (p.ej. 750) aparecen fallos por agotamiento de capacidad del multiplexor, no por el *backlog* de `listen(2)`.

3.2 Backlog de listen vs. clientes conectados

El socket pasivo usa `listen(server, SOMAXCONN)`. Ese backlog es la cola de conexiones **pendientes de accept**, no el número de clientes ya conectados. La cantidad de clientes concurrentes reales se refleja en la métrica **ACT** (conexiones SOCKS5 activas). Confundir ambos conceptos lleva a subestimar o sobrestimar la capacidad del servidor.

3.3 Registro de accesos síncrono

Los logs van a stdout con I/O bloqueante. En localhost el impacto es bajo; bajo carga alta con un filesystem lento podría introducir micro-pausas en el event loop. Un logger asíncrono sería una extensión natural.

3.4 Alcance del protocolo SOCKS

Solo se implementa **CONNECT**. No hay **BIND** ni **UDP ASSOCIATE**. El management escucha solo en IPv4 (**AF_INET**).

4 Posibles extensiones

- **Dissector de credenciales (segunda entrega)**: inspección en COPY de protocolos en texto claro (POP3 como mínimo), estilo ettercap.
- **Soporte BIND y UDP ASSOCIATE**.
- **Logging asincrónico**: buffer en memoria y volcado diferido.
- **Configuración dinámica extendida**: timeouts, tamaño de buffers, tope de conexiones concurrentes.
- **Management dual-stack**: escuchar también en IPv6.
- **Estadísticas avanzadas**: latencia, throughput por usuario, etc.

5 Conclusiones

Se entregó un proxy SOCKS5 monohilo funcional sobre I/O no bloqueante multiplexada, con autenticación RFC 1929 opcional según usuarios registrados, destinos IPv4/IPv6/FQDN, reporte de errores del protocolo, graceful shutdown y un protocolo de management propio con autenticación por secreto compartido.

El diseño alrededor de una máquina de estados por conexión y buffers circulares permitió manejar lecturas/escrituras parciales y el túnel COPY de forma ordenada. Las pruebas de estrés muestran que se cumple el piso de 500 conexiones concurrentes de la consigna; el límite práctico actual está ligado a la capacidad del selector.

Como aprendizaje principal queda la importancia de no introducir round-trips artificiales en el event loop (write optimista) y de documentar con precisión el protocolo de administración: terminadores, autenticación y semántica de métricas frente a conceptos del SO como el backlog de `listen`.

6 Ejemplos de prueba

6.1 Prueba básica del proxy SOCKS5

```
1 $ ./bin/server -A mi_secreto -u testuser:testpass
2 Listening on TCP port 1080
3 Management listening on TCP port 8080
```

Listing 6: Iniciar el servidor

```
1 $ curl --socks5 127.0.0.1:1080 --proxy-user testuser:testpass \
2     http://example.com
```

Listing 7: Proxy con autenticación

```
1 $ ./bin/server -A mi_secreto
2 $ curl --socks5 127.0.0.1:1080 http://example.com
```

Listing 8: Proxy sin usuarios (NO AUTH)

6.2 Prueba de monitoreo

```
1 $ ./bin/client -A mi_secreto get-metrics
2 conexiones activas: 1
3 conexiones historicas: 5
4 bytes transferidos: 24576
5
6 $ ./bin/client -A mi_secreto add-user nuevo pass123
7 usuario agregado
8
9 $ ./bin/client -A mi_secreto del-user nuevo
10 usuario eliminado
```

Listing 9: Métricas y usuarios en caliente

6.3 Prueba de graceful shutdown

```
1 $ kill -TERM <PID>
2 # graceful shutdown: no se aceptan nuevas conexiones, esperando N
   conexiones activas
```

Listing 10: SIGTERM

6.4 Pruebas de estrés

Se midió el proxy en localhost con el harness `scripts/stress.py` (origen TCP local + clientes SOCKS5 concurrentes).

Cuadro 1: Conexiones concurrentes sostenidas

Solicitadas	OK	Fallidas
50	50	0
100	100	0
250	250	0
500	500	0
750	505	245

Hasta 500 conexiones se establecen sin error. Por encima, la tasa de éxito cae al acercarse al tope de descriptors del selector ($\approx$ dos fds por túnel).

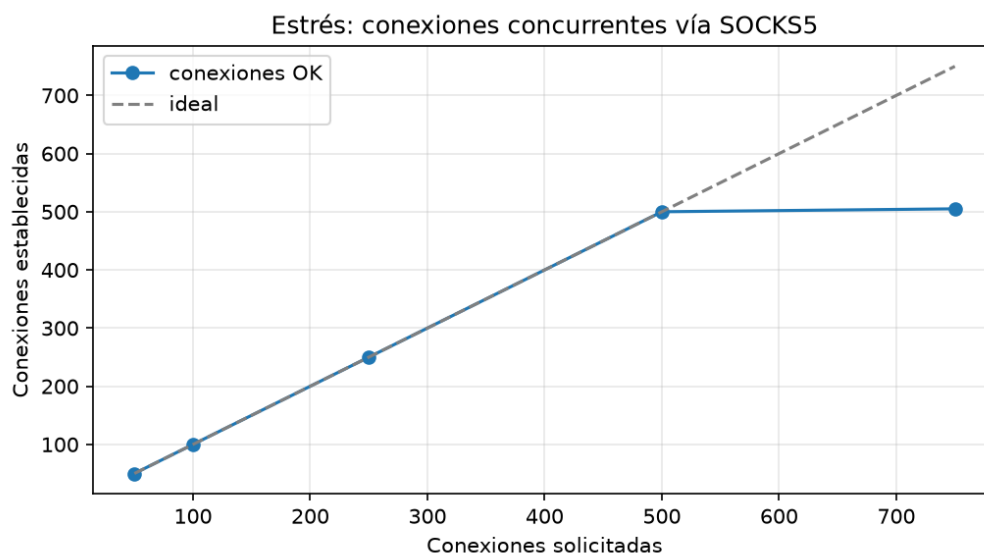


Figura 1: Conexiones concurrentes solicitadas vs. establecidas.

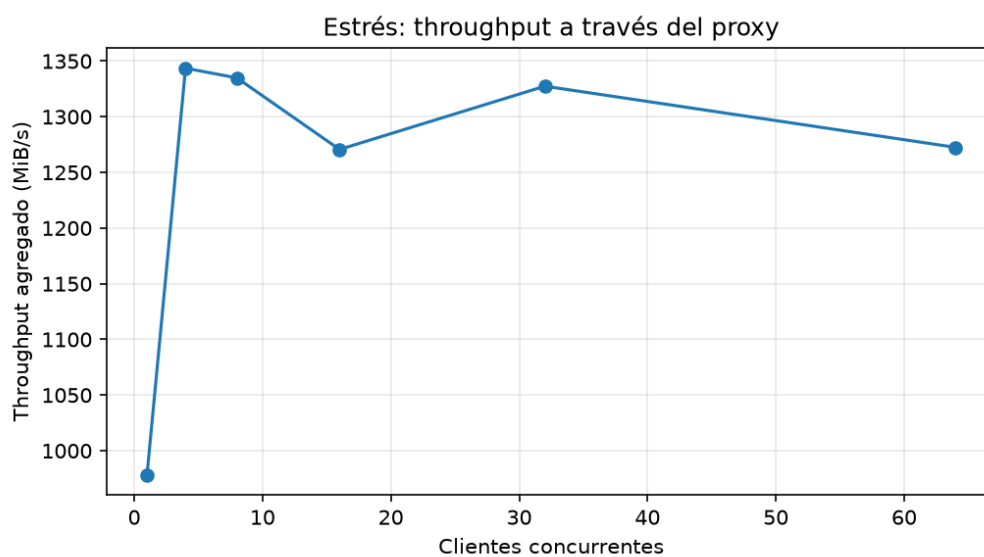


Figura 2: Throughput agregado en loopback (payload 2 MiB por cliente).

En loopback el throughput se mantiene en el orden de cientos/miles de MiB/s y no es el cuello de botella: la degradación relevante aparece en la cantidad de conexiones concurrentes, no en el ancho de banda local. Para reproducir:

```
1 $ bash scripts/run_stress.sh
```

Listing 11: Reproducir estrés

7 Guía de instalación

7.1 Requisitos

- Linux con GCC compatible con C11, GNU Make y pthreads.

7.2 Compilación

```
1 $ make clean && make all
```

Artefactos: bin/server, bin/client. Objetos en obj/.

```
1 $ make clean
```

8 Instrucciones para la configuración

```
1 ./bin/server [OPCIONES]
2
3 -h                Ayuda.
4 -l <dirección>    Bind SOCKS (defecto: dual-stack / 0.0.0.0 vía
                    IPv6).
5 -L <dirección>    Bind management (defecto: 127.0.0.1).
6 -p <puerto>      Puerto SOCKS (defecto: 1080).
7 -P <puerto>      Puerto management (defecto: 8080).
8 -A <secreto>      Secreto de management (defecto: changeme).
9 -u <usr>:<pass>    Usuario inicial (hasta 10).
10 -v               Versión.
```

Listing 12: Opciones del servidor

```
1 ./bin/client [OPCIONES] <comando>
2
3 -L <dirección>    IP management (defecto: 127.0.0.1)
4 -P <puerto>      Puerto management (defecto: 8080)
5 -A <secreto>      Secreto (defecto: changeme)
6
7 get-metrics | add-user <u> <p> | del-user <u> | set-secret <
    nuevo>
```

Listing 13: Opciones del cliente

Los accesos se imprimen en stdout del servidor; para persistirlos:

```
1 $ ./bin/server -A mi_secreto -u admin:admin123 > access.log 2>
   server.err
```

9 Ejemplos de configuración y monitoreo

9.1 Escenario 1: monitoreo local

```
1 $ ./bin/server -A mi_secreto -u admin:admin123
2
3 $ ./bin/client -A mi_secreto get-metrics
4 conexiones activas: 0
5 conexiones historicas: 0
6 bytes transferidos: 0
7
8 $ ./bin/client -A mi_secreto add-user desarrollador pass123
9 usuario agregado
10
11 $ curl --socks5 127.0.0.1:1080 --proxy-user desarrollador:pass123
12 \
13     https://httpbin.org/ip
14
15 $ ./bin/client -A mi_secreto get-metrics
16 conexiones activas: 0
17 conexiones historicas: 1
18 bytes transferidos: 2048
```

9.2 Escenario 2: auditoría de accesos

```
1 $ ./bin/server -A mi_secreto -u admin:admin123 > access.log
2 $ cat access.log
3 [2026-07-12 16:46:17] User: admin | FQDN: example.com | IP
   Destino: 93.184.216.34:80
```

10 Documento de diseño del proyecto

10.1 Diagrama de módulos

```
+-----+
|                               bin/server                               |
| main.c / args.c      socks5nio.c (STM SOCKS5 + COPY)                   |
| hello.c / auth.c / request.c  mng_server.c (STM management)           |
| [infra: selector, buffer, stm, netutils]                               |
+-----+
|                               bin/client                               |
| client.c -- CLI: parsea +OK/-ERR, salida humana                       |
+-----+
```

La infraestructura de multiplexación (`selector`, `buffer`, `stm`) se reutiliza como base; el aporte propio está en el proxy SOCKS5 (`socks5nio` y `parsers`), el protocolo de management y el cliente CLI.

10.2 Componentes propios

10.2.1. Máquina de estados SOCKS5 (`socks5nio.c`)

Implementa el ciclo de vida completo de cada conexión: HELLO, AUTH, REQUEST (resolución + connect no bloqueante), respuesta REP y túnel COPY con write optimista y propagación de `shutdown` parciales.

10.2.2. Parsers SOCKS5

- `hello.c/h`: oferta y selección de métodos.
- `auth.c/h`: usuario/contraseña RFC 1929.
- `request.c/h`: CONNECT con IPv4, IPv6 y FQDN; marshalling de REP.

10.2.3. Servidor de monitoreo (`mng_server.c`)

STM propia (`MNG_READ` → `MNG_EXECUTE` → `MNG_WRITE`) sobre el mismo selector. Auténtica por secreto, muta la tabla de usuarios y expone métricas.

10.2.4. Cliente de monitoreo (`client.c`)

Abstrae el protocolo: no es un netcat. Traduce subcomandos POSIX a líneas del protocolo y presenta resultados legibles con códigos de salida útiles para scripts.

10.3 Flujo de una conexión SOCKS5

Cliente SOCKS5	Servidor Proxy	Servidor Origen
--- HELLO ----->		
<-- METHOD -----		
--- AUTH (opcional) ----->		
<-- STATUS -----		
--- REQUEST CONNECT ----->		
	===== CONNECT =====>	
<-- REP -----		
===== TÚNEL BIDIRECCIONAL (COPY) =====>		